

Modul 08: HttpClient

<i>Ziel:</i>	<i>API Zugriff mittels HttpClient</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~30min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>07.08.2024</i>

1 Web API vorbereiten

In einem neuen Verzeichnis klonen wir als Vorbereitung einen einfachen Node Server und starten diesen:

```
git clone https://github.com/ppedvOrtwin/simple-item-store
npm install
npm run start
```

Die verfügbaren Adressen für den Zugriff sind in der README.md beschrieben und lauten:

- GET <http://localhost:3000/items>
- POST <http://localhost:3000/items>
- PUT <http://localhost:3000/items/{id}>
- DELETE <http://localhost:3000/items/{id}>

Zudem muss im App-Modul das `HttpClientModule` importiert werden. Wenn wir Module verwenden, muss es den Imports in hinzugefügt werden.

```
import { HttpClientModule } from '@angular/common/http';
// ...
imports: [BrowserModule, /* ... */, HttpClientModule],
```

Dann können wir den `HttpClient` mittels Dependency Injection in der entsprechenden Service-Klasse verwenden.

2 Remote Service Klasse

Ein Remote-Task-Service soll mittels asynchronen Methoden und dem HttpClient auf die REST-API zugreifen.

```
const API_URL = 'http://localhost:3000';
const parseDate = <T>(obj: T) => DateTime.fromISO(`${obj}`).toJSDate();
export const parseTask = (task: TaskItem) =>
  ({ ...task, dueDate: parseDate(task.dueDate) });

@Injectable({
  providedIn: 'root',
})
export class TaskService {
  readonly items = signal<TaskItem[]>([]);

  constructor(private http: HttpClient) {}

  loadTasks(): Promise<TaskItem[]> {
    // Observables werden per Konvention mit einem $ suffix gekennzeichnet
    const response$ = this.http.get<TaskItem[]>(`${API_URL}/items`).pipe(
      // Daten konvertieren da Date als string empfangen wird
      map((items) => items.map(parseTask)),
      // mit initialem array starten wenn noch keine Daten empfangen
      startWith(this.items()),
      // Daten in das Signal setzen
      tap((items) => this.items.set(items))
    );
    return lastValueFrom(response$);
  }
}
```

In der Aufgabenliste können wir dann `loadTasks()` aufrufen.

```
export class TaskListComponent implements OnInit {
  constructor(public taskService: TaskService) {}

  async ngOnInit(): Promise<void> {
    await this.taskService.loadTasks();
  }
}
```

Die schreibenden Methoden aktualisieren wir im RemoteTaskService entsprechend und sollten auch die `submit()` Methode in der task-form Komponente anpassen.

```

reload = switchMap(() => this.loadTasks());

async addTask(task: Omit<TaskItem, 'id' | 'completed'>) {
  const newTask = {
    ...task,
    id: `${Date.now()}`,
    completed: false,
  };
  this.items.update((items) => [...items, newTask]);
  return lastValueFrom(this.http.post(`${API_URL}/items`, newTask).
    pipe(this.reload));
}

async updateTask(id: string, task: Partial<TaskItem>) {
  const newTask = { ...this.getTask(id), ...task };
  return lastValueFrom(this.http.put(`${API_URL}/items/${id}`, newTask)
    .pipe(this.reload));
}

async removeTask(id: string) {
  return lastValueFrom(this.http.delete(`${API_URL}/items/${id}`)
    .pipe(this.reload));
}

```

3 WebSocket Verbindung

Wir erstellen noch einen weiteren Service welcher auf eine WebSocket Verbindung zugreift. Die Implementierung dank RxJS ist recht simpel.

```

import { toSignal } from '@angular/core/rxjs-interop';
import { map } from 'rxjs';
import { WebSocket } from 'rxjs/webSocket';

@Injectable({
  providedIn: 'root',
})
export class TaskSocketService {
  readonly subject = WebSocket<TaskItem[]>('ws://localhost:3000')
    .pipe(map((items) => items.map(parseTask)));
  readonly items = toSignal(this.subject);
}

```

Das Observable (subject) vom RxJS Framework wird noch in ein signal konvertiert und schon haben wir unsere stetig aktuelle Liste. Wenn wir diese verwenden und in einem anderen Browser-Fenster nun ein Task hinzufügen, so wird es zeitgleich auch im ursprünglichen Browser-Fenster aktualisiert.

4 Datenquellen auswählen

Wir können noch in der Listen-Komponente Auswahlfelder hinzufügen, um die Datenquelle zu wählen.

```
<div class="mode-select">
  <label> <input type="radio" name="mode" value="local"
    [(ngModel)]="selectedMode" /> Local </label>
  <label> <input type="radio" name="mode" value="rest"
    [(ngModel)]="selectedMode" /> REST </label>
  <label> <input type="radio" name="mode" value="socket"
    [(ngModel)]="selectedMode" /> Sockets </label>
</div>
```

Für das ngModel verwenden wir die neuen Angular Signals.

```
type Mode = 'local' | 'rest' | 'socket';

// Klassendefinition ...

selectedMode = model<Mode>('rest');
tasks = computed(() => {
  switch (this.selectedMode()) {
    case 'local':
      return this.localService.items();

    case 'socket':
      return this.socketService.items();

    default:
      return this.remoteService.items();
  }
});
```

Damit haben wir erfolgreich eine interaktive Todo-App erstellt.